

28 de agosto de 2026.

N1-EXAM – 2026-20

Por favor lea cuidadosamente las siguientes instrucciones:

- Este examen es **individual** y tiene una duración máxima de 80 min.
- Está prohibida cualquier interacción con sus compañeros durante el examen.
- No se permite el uso de dispositivos electrónicos (celulares, relojes inteligentes, agendas electrónicas, computadores, etc.) ni de materiales adicionales como apuntes, libros o impresos.
- En preguntas *abiertas*, debe escribir su respuesta de manera clara y legible, aplicando la sintaxis y los conceptos de Python estudiados en el curso. Se provee una cuadrícula para facilitar la indentación de su código; úsela de manera que quede claro en qué cuadro inicia cada instrucción. **Las respuestas ilegibles o inadecuadamente indentadas, pueden ser calificadas como incorrectas.** Puede usar lápiz, **pero únicamente se aceptarán reclamos sobre respuestas escritas en esfero.** Si daña una hoja de respuestas, al final de este examen se proporciona una hoja vacía con cuadrícula.
- Ante un intento de copia, el profesor podrá **retirarle el enunciado** y asignar una nota de **0.0** o iniciar un **proceso disciplinario**. Por favor, mantenga la mirada exclusivamente en su enunciado. **Los monitores estarán atentos a cualquier intento de fraude.**

Compromiso de integridad:

Al proporcionar sus datos a continuación, usted confirma su compromiso de completar este examen de acuerdo con las normas de integridad académica de la Universidad de los Andes, incluyendo acatar el Reglamento General de Estudiantes de Pregrado y todas las disposiciones relacionadas con evitar el *Fraude académico*¹, establecidas en el Reglamento Disciplinario para Estudiantes de Pregrado.

Nombre: _____

Firma: _____

¹ En el artículo 4 del Reglamento Disciplinario para Estudiantes de Pregrado, se define el *Fraude académico* como “el comportamiento del estudiante que infringe las reglas de la Universidad o las reglas establecidas por el evaluador, en desarrollo de una actividad académica”.

Pregunta 1 (20 pts). La *Fórmula de Herón* permite calcular el área A de un triángulo a partir de la longitud de sus tres lados, sin necesidad de conocer su altura. Primero, se calcula el semiperímetro s como:

$$s = \frac{a + b + c}{2}$$

Conocidos los lados a , b y c del triángulo, junto con su semiperímetro s , el área elevada al cuadrado se obtiene mediante la expresión:

$$A^2 = s(s - a)(s - b)(s - c)$$

Al despejar A tomando la raíz cuadrada, se obtiene la **fórmula de Herón para calcular el área del triángulo**:

$$A = \sqrt{s(s - a)(s - b)(s - c)}$$

Implemente una función en Python que permita calcular la anterior fórmula. **Debe usar `math` para obtener la raíz cuadrada**. Debe implementar únicamente el **cuerpo de la función**. Nombres de variables como a , b , c y s , ya son significativos en este contexto matemático y no deben cambiarse. **Únicamente una implementación 100% correcta de la función obtendrá 20 pts. En cualquier otro caso, aún con errores muy menores, obtendrá 0 pts, sin excepciones.** Por favor verifique que usó `math` correctamente en su respuesta.

```
1 import math
2
3 def calcular_area_heron(a: float, b: float, c: float) -> float:
4     """
5     Calcula el área de un triángulo usando la fórmula de Herón.
6
7     Args:
8         a (float): Longitud del primer lado del triángulo.
9         b (float): Longitud del segundo lado del triángulo.
10        c (float): Longitud del tercer lado del triángulo.
11
12    >>> calcular_area_heron(3, 4, 5)
13    6.0
14    >>> round(calcular_area_heron(2, 2, 2), 4)
15    1.7321
16
17    Returns:
18        float: Área del triángulo.
19        No se aplica ningún redondeo.
20    """
21    s = (a + b + c) / 2 # 2 podría haber sido una constante también
22    return math.sqrt(s * (s - a) * (s - b) * (s - c))
```

La siguiente parte del examen se basa en **N1-PROY: CupiBank**. Aquí un recordatorio de algunas funciones del proyecto que puede asumir que ya están implementadas y puede invocar directamente.

Nombre de la función: `calcular_ingreso_mensual_netto`

Descripción: Calcula el ingreso mensual neto de un cliente.

Parámetro	Tipo	Descripción
<code>salario_base</code>	<code>float</code>	Salario mensual base del cliente.
<code>ingresos_extras</code>	<code>float</code>	Otros ingresos mensuales del cliente (independientes de su salario base).
<code>impuestos</code>	<code>float</code>	Total estimado de impuestos mensuales.

Retorno: `float` — Ingreso mensual neto del cliente, redondeado a dos decimales.

Nombre de la función: `calcular_capacidad_endeudamiento`

Descripción: Calcula la capacidad de endeudamiento de un cliente.

Parámetro	Tipo	Descripción
<code>salario_base</code>	<code>float</code>	Salario mensual base del cliente.
<code>ingresos_extras</code>	<code>float</code>	Otros ingresos mensuales del cliente (independientes de su salario base).
<code>impuestos</code>	<code>float</code>	Total estimado de impuestos mensuales.
<code>gastos_mensuales</code>	<code>float</code>	Total de gastos mensuales del cliente.
<code>porcentaje_endeudamiento</code>	<code>float</code>	Porcentaje del ingreso disponible para nuevas deudas (entre 0.0 y 1.0).

Retorno: `float` — Capacidad de endeudamiento del cliente, redondeada a dos decimales.

Nombre de la función: `calcular_calificacion_crediticia`

Descripción: Calcula la calificación crediticia de un cliente.

Parámetro	Tipo	Descripción
<code>confiabilidad</code>	<code>float</code>	Índice entre 0.0 y 1.0 según historial crediticio del cliente.
<code>referidos</code>	<code>int</code>	Número de clientes referidos (entre 0 y 5).
<code>riesgo</code>	<code>int</code>	Índice de riesgo (1: bajo, 2: medio, 3: alto).

Retorno: `float` — Calificación crediticia del cliente, redondeada a dos decimales.

Nombre de la función: `calcular_cupo_limite`

Descripción: Calcula el cupo límite de un cliente.

Parámetro	Tipo	Descripción
<code>salario_base</code>	float	Salario mensual base del cliente.
<code>ingresos_extras</code>	float	Otros ingresos mensuales del cliente (independientes de su salario base).
<code>impuestos</code>	float	Total estimado de impuestos mensuales.
<code>gastos_mensuales</code>	float	Total de gastos mensuales del cliente.
<code>porcentaje_endeudamiento</code>	float	Porcentaje del ingreso disponible para nuevas deudas (entre 0.0 y 1.0).
<code>confiabilidad</code>	float	Índice entre 0.0 y 1.0 según historial crediticio del cliente.
<code>referidos</code>	int	Número de clientes referidos (entre 0 y 5).
<code>riesgo</code>	int	Índice de riesgo (1: bajo, 2: medio, 3: alto).

Retorno: float — Cupo límite del cliente, redondeado a dos decimales.

Nombre de la función: `calcular_cuota_mensual`

Descripción: Calcula la cuota mensual que un cliente debe pagar por la tarjeta de crédito.

Parámetro	Tipo	Descripción
<code>gasto_actual</code>	float	Monto gastado con la tarjeta de crédito en el mes actual.
<code>saldo_anterior</code>	float	Saldo total pendiente de meses anteriores.
<code>numero_pagos</code>	int	Número de meses para amortizar el saldo anterior (entre 1 y 48).

Retorno: float — Cuota mensual del cliente, redondeada a dos decimales.

Nombre de la función: `calcular_tasa_efectiva_anual`

Descripción: Calcula la tasa de interés efectiva anual equivalente a una tasa de interés mensual.

Parámetro	Tipo	Descripción
<code>tasa_mensual</code>	float	Tasa de interés mensual expresada como decimal (ej. 0.025 para 2.5 %).

Retorno: float — Tasa de interés efectiva anual equivalente (12 meses), redondeada a cuatro decimales.

Pregunta 2 (20 pts). Implemente la función `calcular_plazo_pago` en Python que permita calcular el número de pagos mensuales necesarios para cancelar un saldo pendiente, dado un valor de cuota fija y una tasa de interés mensual. Se aclara que esta función era parte de las funciones solicitadas en **N1-PROY**.

$$numero_{pagos} = \frac{\ln\left(\frac{cuota_{fija}}{cuota_{fija} - tasa_{interes_mensual} \cdot saldo_{pendiente}}\right)}{\ln(1 + tasa_{interes_mensual})}$$

Donde:

- `saldo_pendiente`: Saldo total pendiente a amortizar.
- `cuota_fija`: Valor fijo de la cuota mensual que el cliente puede pagar. Debe ser mayor al interés mensual generado por `saldo_pendiente` para que el resultado sea válido.
- `tasa_interes_mensual`: Tasa de interés mensual aplicada al saldo, expresada como decimal. No puede ser 0.0 ni menor o igual a -1.0.

Rúbrica de calificación:

Descripción	Cumple	Cumple parcialmente	Incumple
(16 pts) Calcula correctamente el número de pagos mensuales, aplicando exactamente la fórmula dada y usando correctamente a la función <code>log</code> .	16		0
(2 pts) Redondea el resultado a la cantidad de decimales indicada en la documentación.	2	0	0
(2 pts) El valor retornado corresponde al tipo de dato especificado en la signatura.	2	0	0
Total	_____ / 20		

Observaciones (espacio para quien califica):

Nota: En la siguiente hoja, debe implementar únicamente el **cuerpo** de la función. La signatura, la documentación y los `imports` necesarios ya se encuentran dados. No debe crear doctests.

```

1  from math import log
2
3  def calcular_plazo_pago(saldo_pendiente: float, cuota_fija: float, tasa_interes_mensual: float) -> float:
4      """
5      Calcula el número de pagos mensuales necesarios para cancelar un saldo pendiente, dado un valor de
6      cuota fija y una tasa de interés mensual.
7
8      Args:
9          saldo_pendiente (float): Saldo total pendiente a amortizar.
10         cuota_fija (float): Valor fijo de la cuota mensual que
11             el cliente puede pagar. Debe ser mayor al interés mensual generado por saldo_pendiente
12             (cuota_fija > tasa_interes_mensual * saldo_pendiente) para que el resultado sea válido.
13         tasa_interes_mensual (float): Tasa de interés mensual
14             aplicada al saldo, expresada como decimal. No puede ser 0.0 ni menor o igual a -1.0.
15
16     Returns:
17         float: Número de pagos mensuales necesarios para cancelar el saldo, redondeado a dos decimales.
18     """
19     factor = cuota_fija / (cuota_fija - tasa_interes_mensual * saldo_pendiente)
20     numero_pagos = log(factor) / log(1 + tasa_interes_mensual)
21
22     return round(numero_pagos, 2)

```

Pregunta 3 (30 pts). Implemente la función `calcular_monto_maximo_financiable` en Python que permita determinar cuánto dinero podría financiar como máximo un cliente, conociendo cuánto puede pagar mensualmente, la tasa de interés y el plazo deseado. Debe utilizar la siguiente fórmula:

$$\text{monto}_{\text{maximo}} = \text{capacidad}_{\text{endeudamiento}} \cdot \frac{1 - e^{-\text{numero_pagos} \cdot \ln(1 + \text{tasa}_{\text{interes_mensual}})}}{\text{tasa}_{\text{interes_mensual}}}$$

Donde:

- `numero_pagos`: Número de pagos mensuales (plazo) que el cliente desea utilizar para pagar el monto financiado.
- `tasa_interes_mensual`: Tasa de interés mensual aplicada al monto financiado, expresada como decimal. No puede ser 0.0 ni menor o igual a -1.0.

Rúbrica de calificación:

Descripción	Cumple	Cumple parcialmente	Incumple
(4 pts) Escribió correctamente la signature de la función.	4		0
(18 pts) Calcula correctamente el monto máximo financiable, aplicando exactamente la fórmula y usando correctamente las funciones <code>exp</code> y <code>log</code> .	18		0
(5 pts) Reutiliza la función preexistente acorde, mediante composición, sin reimplementar su lógica.	5	0	0
(2 pts) Redondea el resultado a la cantidad de decimales indicada en la documentación.	2	0	0
(1 pt) El valor retornado corresponde al tipo de dato especificado en la documentación dada (docstring).	1	0	0
Total	_____ / 30		

Observaciones (espacio para quien califica):

Nota: En la siguiente hoja, debe implementar la **signature** (con `def`) y el **cuerpo** de la función. La documentación y los `imports` necesarios ya se encuentran dados. No debe crear doctests.

```

1  from math import exp
2
3  def calcular_monto_maximo_financiable(
4      salario_base: float,
5      ingresos_extras: float,
6      impuestos: float,
7      gastos_mensuales: float,
8      porcentaje_endeudamiento: float,
9      numero_pagos: int,
10     tasa_interes_mensual: float
11 ) -> float:
12     """
13     Calcula el monto máximo que un cliente podría financiar utilizando su capacidad de endeudamiento
14     como cuota mensual.
15
16     Args:
17         salario_base (float): Salario mensual base del cliente.
18         ingresos_extras (float): Otros ingresos mensuales del cliente (independientes de su salario base).
19         impuestos (float): Total estimado de impuestos mensuales.
20         gastos_mensuales (float): Total de gastos mensuales del cliente.
21         porcentaje_endeudamiento (float): Porcentaje del ingreso disponible para nuevas deudas
22         (entre 0.0 y 1.0).
23         numero_pagos (int): Número de pagos mensuales (plazo) que el cliente desea utilizar para pagar
24         el monto financiado.
25         tasa_interes_mensual (float): Tasa de interés mensual aplicada al monto financiado, expresada como
26         decimal. No puede ser 0.0 ni menor o igual a -1.0.
27
28     >>> calcular_indice_riesgo_ajustado(0.05, 0.02, 0.16) # Para rendimiento_esperado 0.05,
29         tasa_libre_riesgo_mensual 0.02 y varianza_portafolio 0.16.
30     -0.55
31
32     Returns:
33         float: Monto máximo que el cliente podría financiar, redondeado a dos decimales.
34     """
35     capacidad_endeudamiento = calcular_capacidad_endeudamiento(
36         salario_base, ingresos_extras, impuestos,
37         gastos_mensuales, porcentaje_endeudamiento
38     )
39     factor_descuento = exp(-numero_pagos * log(1 + tasa_interes_mensual))
40     factor_anualidad = (1 - factor_descuento) / tasa_interes_mensual
41     monto_maximo = capacidad_endeudamiento * factor_anualidad
42
43     return round(monto_maximo, 2)

```

Pregunta 4 (30 pts). Implemente la función `calcular_indice_rendimiento_riesgo` en Python, que permite calcular un índice de rendimiento ajustado por riesgo para un portafolio. Debe utilizar la siguiente fórmula:

$$indice_{rendimiento_riesgo} = \frac{\ln\left(\frac{1 + rendimiento_{esperado}}{1 + tasa_efectiva_anual(tasa_{libre_riesgo_mensual})}\right)}{\sqrt{varianza_{portafolio}}}$$

Donde:

- `rendimiento_esperado`: Rendimiento anual esperado del portafolio, expresado como decimal.
- `tasa_libre_riesgo_mensual`: Tasa mensual libre de riesgo, expresada como decimal.
- `varianza_portafolio`: Varianza histórica de los rendimientos del portafolio.

Rúbrica de calificación:

Descripción	Cumple	Cumple parcialmente	Incumple
(22 pts) Calcula correctamente el índice de rendimiento ajustado por riesgo, aplicando exactamente la fórmula dada y usando correctamente las funciones <code>log</code> y <code>sqrt</code> .	22		0
(5 pts) Reutiliza la función preexistente acorde, mediante composición, sin reimplementar su lógica.	5	0	0
(2 pts) Redondea el resultado a la cantidad de decimales indicada en la documentación.	2	0	0
(1 pt) El valor retornado corresponde al tipo de dato especificado en la signatura.	1	0	0
Total	_____ / 30		

Observaciones (espacio para quien califica):

Nota: En la siguiente hoja, debe implementar únicamente el **cuerpo** de la función. La signatura, la documentación y los `imports` necesarios ya se encuentran dados. No debe crear doctests.

```

1  from math import log, sqrt
2
3  def calcular_indice_rendimiento_riesgo(
4      rendimiento_esperado: float,
5      tasa_libre_riesgo_mensual: float,
6      varianza_portafolio: float
7  ) -> float:
8      """
9      Calcula un índice de rendimiento ajustado por riesgo para un portafolio.
10
11      Args:
12          rendimiento_esperado (float): Rendimiento anual esperado del portafolio,
13              expresado como decimal.
14          tasa_libre_riesgo_mensual (float): Tasa mensual libre de riesgo, expresada como decimal.
15          varianza_portafolio (float): Varianza histórica de los rendimientos del portafolio.
16
17      Returns:
18          float: Índice de rendimiento ajustado por riesgo, redondeado a dos decimales.
19      """
20      tasa_libre_riesgo_anual = calcular_tasa_efectiva_anual(tasa_libre_riesgo_mensual)
21
22      volatilidad = sqrt(varianza_portafolio)
23
24      exceso_rendimiento = log(
25          (1 + rendimiento_esperado) /
26          (1 + tasa_libre_riesgo_anual)
27      )
28
29      indice = exceso_rendimiento / volatilidad
30
31      return round(indice, 2)

```